

Caracterização Experimental do Limiar de Cache L3 em Throughput de Memória: um Estudo em Arquitetura Híbrida Intel Raptor Lake

André Borges Viana¹

¹Curso de Ciência da Computação – CESAR School
Recife – PE – Brasil

andre.viana@cesar.school

Abstract. *This paper experimentally characterizes the effect of L3 cache capacity on memory copy throughput in an Intel Core i5-13420H (Raptor Lake) hybrid processor. Using the mbw benchmarking tool across array sizes spanning the 12 MiB L3 boundary, combined with spatial locality tests and a comparison against NVMe storage I/O, we identify a precise 17.7% bandwidth drop coincident with the cache’s nominal capacity. We further show that hardware prefetching substantially mitigates the penalty of non-sequential access patterns, and discuss the methodological limitations imposed by WSL2 virtualization on benchmark fidelity.*

Resumo. *Este artigo caracteriza experimentalmente o efeito da capacidade do cache L3 sobre o throughput de cópia de memória em um processador híbrido Intel Core i5-13420H (Raptor Lake). Utilizando a ferramenta de benchmark mbw em tamanhos de array que atravessam o limiar de 12 MiB do L3, combinada a testes de localidade espacial e a uma comparação com I/O de armazenamento NVMe, identificamos uma queda precisa de 17,7% na largura de banda coincidente com a capacidade nominal do cache. Mostramos ainda que o prefetching de hardware mitiga substancialmente a penalidade de padrões de acesso não sequenciais, e discutimos as limitações metodológicas impostas pela virtualização WSL2 sobre a fidelidade do benchmark.*

1. Introdução

A eficiência com que um sistema computacional movimenta dados entre a unidade central de processamento e a memória principal é, com frequência, o fator determinante do desempenho observado em workloads reais, superando, em muitos casos, a influência da frequência de clock ou do número de núcleos disponíveis. Esse fenômeno, conhecido na literatura como *memory wall*, decorre da disparidade crescente entre a velocidade de processamento das CPUs modernas e a latência de acesso à memória RAM, disparidade que a hierarquia de cache (L1, L2 e L3) existe justamente para mitigar.

Compreender o comportamento dessa hierarquia em hardware real, e não apenas em modelos teóricos, é uma competência central para o desenvolvimento de software de alto desempenho, da otimização de bancos de dados ao design de algoritmos de computação científica. Em particular, processadores de arquitetura híbrida – que combinam núcleos de alto desempenho (P-cores) e núcleos de eficiência energética (E-cores) em um mesmo *die*, como é o caso da linha Intel Raptor Lake – introduzem nuances

adicionais à hierarquia de memória, uma vez que diferentes tipos de núcleo apresentam capacidades de cache L1 e L2 distintas, ainda que compartilhem um L3 unificado.

Este trabalho investiga experimentalmente o seguinte problema: qual é o efeito quantitativo da capacidade do cache L3 sobre o throughput de cópia de memória em um processador híbrido Intel Core i5-13420H, e em que medida esse efeito se manifesta como um limiar identificável e reproduzível? A motivação para essa investigação surge da constatação de que, embora a teoria preveja qualitativamente uma queda de desempenho quando o conjunto de dados excede a capacidade do último nível de cache, a magnitude exata dessa queda – e sua correspondência precisa com as especificações nominais do fabricante – é uma questão empírica que requer validação experimental direta.

Para isso, foram conduzidos experimentos de medição de largura de banda de memória com a ferramenta mbw, variando sistematicamente o tamanho dos arrays processados em relação à capacidade do cache L3 (12 MiB), complementados por testes de localidade espacial e por uma caracterização da posição da RAM na hierarquia mais ampla de armazenamento, com base em medições de I/O com a ferramenta fio. A análise correlaciona os resultados experimentais com a topologia de hardware identificada através de múltiplas ferramentas de inspeção (`lscpu`, CPU-Z e `lstopo`), reconciliando divergências entre essas fontes quando necessário.

As contribuições deste trabalho são as seguintes: (i) a caracterização experimental precisa do limiar de transição de desempenho associado ao cache L3 em um processador Raptor Lake; (ii) a quantificação do efeito de localidade espacial em presença de *prefetching* de hardware agressivo, evidenciando que modelos simplificados de penalidade de *miss* podem subestimar a eficácia de mecanismos de antecipação modernos; e (iii) a discussão metodológica das limitações impostas por ambientes de virtualização leve (WSL2) sobre a fidelidade de microbenchmarks de memória, um ponto frequentemente negligenciado em estudos didáticos de arquitetura de computadores.

O restante deste artigo está organizado da seguinte forma: a Seção 2 discute trabalhos relacionados; a Seção 3 apresenta a fundamentação teórica necessária; a Seção 4 descreve a metodologia experimental; a Seção 5 apresenta os resultados obtidos; a Seção 6 discute esses resultados e suas implicações; e a Seção 7 conclui o trabalho e aponta direções futuras.

2. Trabalhos Relacionados

A caracterização experimental de hierarquias de memória é um tema consolidado na literatura de arquitetura de computadores. Drepper [1] apresenta uma das referências mais difundidas sobre o assunto, detalhando como o comportamento do cache impacta o desempenho de software e propondo benchmarks de referência para medir efeitos de localidade espacial e temporal. Sua análise inclui experimentos com varredura sistemática de tamanho de *working set*, técnica empregada também neste trabalho, e demonstra que a curva de latência por elemento acessado apresenta patamares (*plateaus*) bem definidos, cada um correspondendo a um nível distinto da hierarquia de cache. Embora seu trabalho seja anterior ao surgimento de arquiteturas híbridas, os princípios que estabelece – particularmente a relação entre tamanho de *working set* e taxa de *miss* – permanecem diretamente aplicáveis e constituem a base teórica deste estudo.

Hennessey e Patterson [2] sistematizam o tratamento quantitativo da hierarquia

de memória, formalizando os conceitos de penalidade de *miss*, taxa de acerto e suas implicações sobre o tempo médio de acesso à memória (AMAT), modelo detalhado na Seção 3.2 e utilizado como arcabouço interpretativo dos resultados experimentais apresentados neste trabalho. Os autores também discutem extensivamente o papel do *prefetching* de hardware como técnica de ocultação de latência, fundamentando a discussão da Seção 6 sobre o comportamento observado no teste de localidade espacial.

Estudos voltados especificamente para arquiteturas multiprocessadas, como o de Molka et al. [3], investigam efeitos de coerência de cache e contenção de banda em processadores multinúcleo Intel Nehalem, utilizando metodologia experimental semelhante à empregada aqui – variação sistemática do tamanho do conjunto de dados para identificar limiares de transição entre níveis de cache, além de medições de banda agregada sob múltiplos threads concorrentes. Os autores demonstram que, em arquiteturas com múltiplos controladores de memória (NUMA), a banda observada depende fortemente do posicionamento dos dados em relação ao núcleo que os acessa – uma variável de complexidade adicional que não se aplica ao processador de *single-socket* e NUMA único investigado no presente estudo, mas que ilustra a importância de caracterizar cada arquitetura individualmente, em vez de extrapolar resultados de uma geração de processador para outra.

Diferentemente desses trabalhos, que se concentram em arquiteturas homogêneas de servidor, o presente estudo aborda uma arquitetura híbrida P+E de uso móvel/notebook, contexto em que a literatura experimental é comparativamente mais escassa. A introdução de núcleos heterogêneos pela Intel a partir da 12^a geração (Alder Lake) e mantida na 13^a geração (Raptor Lake, objeto deste estudo) altera a relação entre tipo de núcleo e capacidade de cache de forma que merece caracterização própria: como detalhado na Seção 3.3, P-cores e E-cores possuem capacidades de L1 e L2 distintas, ainda que compartilhem o mesmo L3 – assimetria que não existe em processadores homogêneos como os estudados por Molka et al.

Por fim, ressalta-se que a maior parte da literatura disponível assume execução em hardware *bare-metal*. Este trabalho discute explicitamente o impacto de uma camada de virtualização leve (WSL2) sobre a fidelidade dos resultados, contribuição que tem ganhado relevância crescente diante da popularização de ambientes de desenvolvimento baseados em máquinas virtuais leves e contêineres, nos quais desenvolvedores frequentemente conduzem testes de desempenho sem necessariamente estarem cientes do *overhead* introduzido pela camada de virtualização sobre operações sensíveis a latência, como acessos a memória e armazenamento.

3. Fundamentação Teórica

3.1. Hierarquia de Memória

A hierarquia de memória é uma organização em camadas que equilibra custo, capacidade e velocidade de acesso. Os níveis mais próximos da CPU – registradores, L1, L2 e L3 – oferecem latências na ordem de nanossegundos, porém com capacidade limitada. A RAM principal, embora com capacidade muito superior, possui latências dezenas de vezes maiores. O princípio fundamental que justifica essa arquitetura é a localidade de referência: programas tendem a acessar, em intervalos curtos, os mesmos dados (localidade temporal) ou dados em endereços próximos (localidade espacial) [2].

Quando um dado solicitado pela CPU não está no cache (*cache miss*), o controlador de memória precisa buscá-lo no nível inferior da hierarquia, incorrendo no chamado *miss penalty*. Esse custo adicional reduz o throughput efetivo e aumenta a latência percebida pela aplicação. A taxa de *miss* depende diretamente do tamanho do conjunto de dados em relação à capacidade do cache: *datasets* menores que o cache de determinado nível tendem a apresentar alta taxa de acerto e, conseqüentemente, alta largura de banda.

3.2. Tempo Médio de Acesso à Memória (AMAT)

Hennessy e Patterson [2] formalizam o impacto agregado da hierarquia de memória através da métrica de Tempo Médio de Acesso à Memória (*Average Memory Access Time*, AMAT), definida recursivamente como:

$$AMAT = T_{\text{acerto}} + \text{Taxa}_{\text{miss}} \times \text{Penalidade}_{\text{miss}} \quad (1)$$

em que T_{acerto} é o tempo de acesso em caso de acerto no nível considerado, $\text{Taxa}_{\text{miss}}$ é a fração de acessos que não encontram o dado nesse nível, e $\text{Penalidade}_{\text{miss}}$ é o custo adicional de buscar o dado no nível imediatamente inferior – que, por sua vez, pode ser expandido recursivamente caso esse nível também não contenha o dado. Em uma hierarquia de três níveis de cache mais RAM, a penalidade de *miss* do L3 corresponde essencialmente à latência de acesso à RAM, tipicamente uma a duas ordens de grandeza superior à latência do L1.

Esse modelo prevê, de forma direta, o comportamento observado experimentalmente neste estudo: enquanto o conjunto de dados manipulado cabe inteiramente no L3, a $\text{Taxa}_{\text{miss}}$ desse nível tende a zero (excetuando-se os *misses* de carga inicial, ou *cold misses*), e o AMAT se aproxima do tempo de acesso ao L3. No momento em que o conjunto de dados excede a capacidade do L3, a $\text{Taxa}_{\text{miss}}$ cresce abruptamente, e o termo $\text{Taxa}_{\text{miss}} \times \text{Penalidade}_{\text{miss}}$ passa a dominar o AMAT, produzindo justamente o tipo de transição abrupta de desempenho investigada neste trabalho.

É importante notar que a relação entre throughput e AMAT é inversa: $\text{throughput} \propto 1/\text{AMAT}$. Assim, a queda percentual no throughput observada empiricamente (Seção 5) pode ser interpretada como uma estimativa direta do aumento percentual no AMAT efetivo ao se cruzar o limiar de capacidade do L3, ainda que fatores como paralelismo de acesso à memória (múltiplas requisições em voo) e o comportamento do controlador de memória DDR5 introduzam não linearidades que o modelo AMAT simplificado não captura integralmente.

3.3. Arquitetura Híbrida Raptor Lake

O Intel Core i5-13420H é um processador móvel de 13ª geração baseado na microarquitetura Raptor Lake. Sua configuração híbrida combina 4 núcleos P-core (*Performance-core*, baseados na microarquitetura Raptor Cove), cada um com suporte a Hyper-Threading (8 threads no total), com 4 núcleos E-core (*Efficient-core*, baseados em Gracemont), sem Hyper-Threading (4 threads), totalizando 8 núcleos físicos e 12 threads lógicas. Essa topologia foi confirmada via CPU-Z e é detalhada na Seção 4.

Esse modelo de design, introduzido pela Intel a partir da 12ª geração (microarquitetura Alder Lake) e mantido na 13ª geração (Raptor Lake, objeto deste

estudo), é inspirado em arquiteturas *big.LITTLE* originalmente popularizadas em processadores móveis ARM. A premissa de design é que cargas de trabalho de pico (*burst*) sejam direcionadas a P-cores, que maximizam o desempenho por *thread* ao custo de maior consumo energético e área de *die*, enquanto cargas de trabalho em segundo plano ou de baixa intensidade sejam direcionadas a E-cores, que oferecem desempenho por *thread* reduzido mas com eficiência energética e densidade de área substancialmente superiores. O escalonador do sistema operacional, em conjunto com o microcontrolador *Intel Thread Director* embarcado no processador, é responsável por essa atribuição dinâmica em tempo de execução, com base em heurísticas de prioridade e intensidade de uso observada de cada *thread*.

A hierarquia de cache desse processador é estruturada de forma assimétrica entre os dois tipos de núcleo: cada P-core possui 48 KB de L1d e 32 KB de L1i dedicados, além de 1.280 KB de L2 também dedicado por núcleo; os quatro E-cores, por sua vez, compartilham um único bloco de L2 de 2.048 KB, com L1d de 32 KB e L1i de 64 KB por núcleo. O L3, de 12 MiB, é unificado e compartilhado entre todos os oito núcleos, representando o último nível de cache antes da interface com a RAM DDR5. O processador também suporta instruções SIMD AVX2 e AVX-VNNI, relevantes para workloads de processamento vetorial.

Essa assimetria de L2 tem implicações diretas sobre o comportamento de cache observável experimentalmente: um *workload* executado em um único P-core dispõe de um L2 dedicado de 1.280 KB sem contenção de outros núcleos, ao passo que um *workload* executado em um E-core compete, potencialmente, pelos mesmos 2.048 KB de L2 com até outros três E-cores simultaneamente ativos. Embora o L3 seja unificado e, portanto, não esteja sujeito a essa mesma assimetria, a capacidade efetiva de cache disponível para um *thread* antes de atingir o L3 varia de forma não trivial dependendo do tipo de núcleo ao qual o sistema operacional o atribui – um fator de confusão (*confound*) relevante para a interpretação de qualquer benchmark de memória executado nessa classe de processador, discutido com mais detalhe na Seção 6.1.

3.4. Localidade Espacial e Prefetching de Hardware

A localidade espacial refere-se à tendência de um programa acessar, após um dado endereço, endereços fisicamente próximos na memória. Algoritmos que percorrem arrays de forma sequencial (*row-major* em matrizes) se beneficiam do *prefetching* de hardware, mecanismo que antecipa a carga de linhas de cache adjacentes antes mesmo de serem solicitadas. Em contraste, acessos com *stride* grande (*column-major* em matrizes armazenadas em *row-major*) tendem a gerar *misses* mais frequentes, pois cada acesso salta o equivalente a múltiplas linhas de cache (tipicamente 64 bytes cada). No entanto, prefetchers modernos são capazes de detectar padrões de *stride* regular – mesmo quando não sequenciais – e antecipar parcialmente os acessos subsequentes, atenuando o impacto teórico previsto por modelos mais simples [1].

4. Metodologia

4.1. Ambiente Experimental

Todos os experimentos foram conduzidos no ambiente descrito na Tabela 1. A topologia de núcleos foi reconciliada entre três ferramentas de inspeção: o utilitário `lscpu`, que

reportou “6 instâncias” de L1/L2 (uma contagem agregada por módulo físico, não por tipo de núcleo); o `lstopo`, que apresentou uma topologia simplificada de 6 grupos de núcleo; e o CPU-Z, que forneceu a granularidade mais detalhada, confirmando explicitamente a divisão real de 4 P-cores (8 threads via Hyper-Threading) e 4 E-cores (4 threads, sem Hyper-Threading). Adotou-se a topologia do CPU-Z como referência por ser a fonte mais granular e explícita quanto à distinção entre tipos de núcleo.

Table 1. Configuração do ambiente experimental

Componente	Especificação
Processador	Intel Core i5-13420H (Raptor Lake)
Núcleos / Threads	8 físicos (4P+4E) / 12 lógicas
Cache L1d / L1i (P-core)	48 KB / 32 KB por núcleo
Cache L1d / L1i (E-core)	32 KB / 64 KB por núcleo
Cache L2 (P-core)	1.280 KB dedicado por núcleo
Cache L2 (E-core)	2.048 KB compartilhado (4 núcleos)
Cache L3	12 MiB (unificado, compartilhado)
RAM	16 GB DDR5
Armazenamento	NVMe SSD (512 GB)
Sistema operacional	Ubuntu 24.04 LTS via WSL2 (Windows 11)

4.2. Ferramentas e Procedimentos

O benchmark de throughput de memória foi conduzido com a ferramenta `mbw` (*Memory Bandwidth benchmark*), que realiza operações de cópia de arrays de tamanho configurável e reporta a largura de banda em MiB/s. Foram selecionados três tamanhos de array estratégicos: 16 MiB (próximo ao limiar do L3 de 12 MiB), 128 MiB (substancialmente maior que o L3, forçando acesso predominante à RAM) e 1.024 MiB (workload representativo de aplicações reais com grandes *datasets*). Cada medição foi executada com o método `MEMCPY` do `mbw`.

Para a análise de localidade espacial, foi utilizado um script Python que percorre uma matriz quadrada de *floats* de 64 bits, de dimensão 6.000×6.000 (aproximadamente 274,7 MB), de duas formas: Loop A, com acesso sequencial linha por linha (*row-major*, favorável ao *prefetcher*), e Loop B, com acesso coluna por coluna (*column-major*, gerando saltos regulares na memória). O tempo de execução de cada loop foi medido com `time.perf_counter()` e comparado pela razão B/A.

Para posicionar a RAM na hierarquia mais ampla do sistema de armazenamento, foram conduzidos testes de I/O com a ferramenta `fio`, em dois padrões: leitura sequencial com blocos de 1 MiB (representativo de transferências em lote) e leitura aleatória com blocos de 4 KiB (representativo de acesso a pequenos registros). Esses testes permitem comparar o throughput de memória obtido via `mbw` com a vazão real de um dispositivo NVMe, contextualizando a magnitude dos números de banda de memória frente a outros componentes do sistema.

A coleta das especificações de cache e topologia foi realizada com `lscpu`, `cat /proc/cpuinfo`, `lstopo` e CPU-Z (Windows, fora do WSL2). A identificação dos parâmetros de memória (DDR5, capacidade, uso) foi feita via `/proc/meminfo`.

4.3. Desenho Experimental e Critérios de Seleção dos Tamanhos de Array

A escolha dos três tamanhos de array utilizados no benchmark de throughput (16, 128 e 1.024 MiB) seguiu um critério de posicionamento deliberado em relação à capacidade do L3 (12 MiB), e não uma amostragem uniforme do espaço de tamanhos possíveis. O valor de 16 MiB foi selecionado por estar imediatamente acima da capacidade nominal do L3, permitindo observar o início da transição de regime sem se afastar excessivamente do limiar. O valor de 128 MiB foi selecionado por exceder o L3 em mais de uma ordem de grandeza, eliminando qualquer contribuição residual do cache e isolando o comportamento de banda em regime de RAM. Por fim, 1.024 MiB foi incluído como ponto de controle para verificar se a banda em regime de RAM permanece estável à medida que o tamanho do *dataset* aumenta ainda mais – hipótese que, como discutido na Seção 5.1, foi confirmada experimentalmente.

Essa estratégia de amostragem, embora suficiente para identificar a existência e a magnitude do limiar de transição, não permite caracterizar com precisão a forma exata da curva entre 12 e 128 MiB – limitação discutida formalmente na Seção 6.2 como ameaça à validade interna do estudo.

4.4. Repetibilidade

Os testes de `mbw` foram executados de forma sequencial, sem outras cargas de trabalho concorrentes intencionalmente introduzidas no sistema, embora o ambiente WSL2 não ofereça isolamento completo de processos do sistema hospedeiro Windows. Os testes de localidade espacial incluíram uma fase de aquecimento (*warm-up*) anterior à medição, para reduzir a influência de *cold misses* e de efeitos de inicialização do interpretador Python sobre o tempo medido.

5. Resultados

5.1. Throughput de Memória por Tamanho de Array

A Figura 1 apresenta os valores de largura de banda observados para os três tamanhos de array testados. A transição de 16 MiB para 128 MiB representa a queda mais expressiva: –17,7% no throughput (de 10.684 MiB/s para 8.797 MiB/s). O throughput entre 128 MiB e 1.024 MiB se estabiliza (8.797 vs. 8.991 MiB/s), evidenciando que, uma vez esgotada a capacidade do L3, o sistema opera em regime de banda limitada pela RAM.

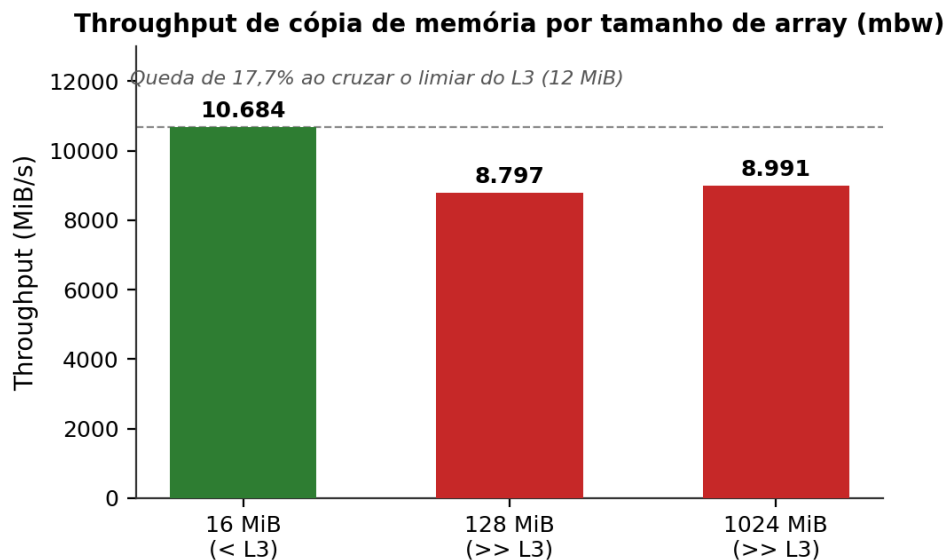


Figure 1. Throughput de cópia de memória medido pelo mbw, evidenciando a queda no cruzamento do limiar de 12 MiB do cache L3.

Esse comportamento é coerente com a capacidade do L3 (12 MiB): ao operar com array de 16 MiB, parte dos dados já excede o L3 e precisa ser buscada na RAM, mas o *prefetcher* e o reaproveitamento de linhas de cache ainda atenuam parcialmente o impacto. Ao passar para 128 MiB, o *dataset* supera em mais de dez vezes a capacidade do L3, e a quase totalidade dos acessos incorre em *miss* no último nível de cache, forçando acesso constante à RAM.

5.2. Localidade Espacial

O teste de localidade espacial revelou uma diferença mensurável, porém modesta, entre os dois padrões de acesso, conforme apresentado na Figura 2.

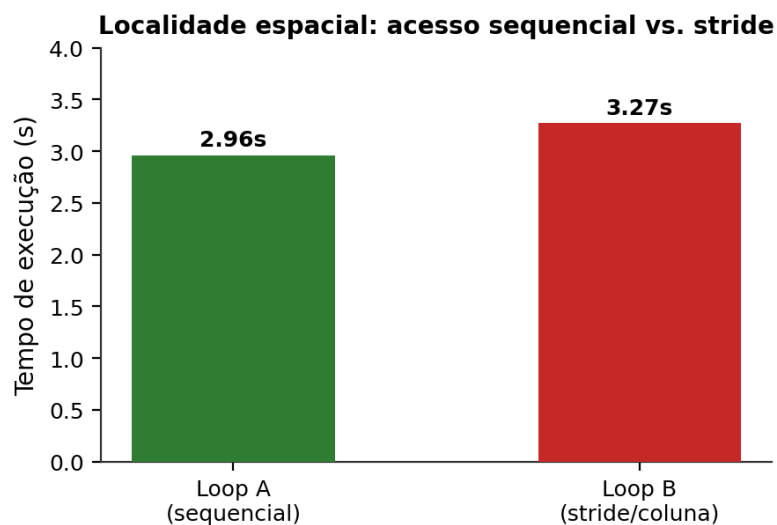


Figure 2. Comparação de tempos de execução entre acesso sequencial (Loop A) e acesso em stride (Loop B).

O acesso em *stride* (Loop B) resultou em tempo 11% superior ao sequencial (Loop A: 2,96s; Loop B: 3,27s; razão B/A = 1,11×). Essa razão é notavelmente baixa frente ao que modelos simplificados de penalidade de *miss* preveriam para um padrão de acesso column-major em uma matriz que excede largamente a capacidade do L1 e do L2. Esse resultado é discutido na Seção 6 à luz da capacidade de *prefetching* dos núcleos Raptor Cove.

5.3. Posição da RAM na Hierarquia de Armazenamento

A Figura 3 contextualiza os números de throughput de memória obtidos frente à vazão real de armazenamento, medida via `fio`. A leitura sequencial em blocos de 1 MiB atingiu 2.526 MB/s, enquanto a leitura aleatória em blocos de 4 KiB atingiu apenas 45 MB/s, com latência média de 90,5 microssegundos por operação.

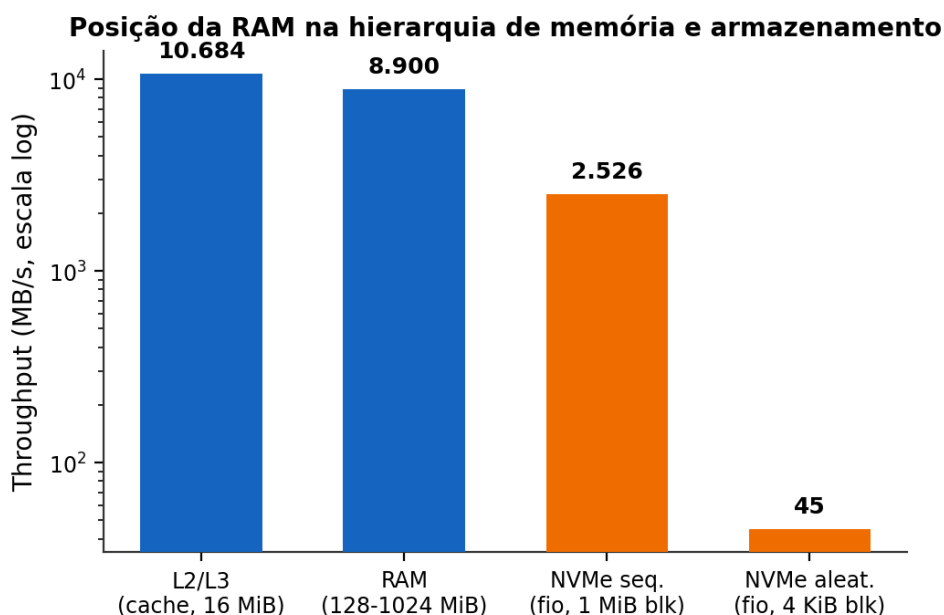


Figure 3. Throughput em escala logarítmica, comparando cache/RAM (mbw) e armazenamento NVMe (fio) sequencial e aleatório.

A diferença entre os dois extremos da hierarquia é substancial: mesmo a RAM em regime de banda reduzida (pós-L3) é aproximadamente 3,5 vezes mais rápida que a leitura sequencial em NVMe, e quase 200 vezes mais rápida que a leitura aleatória em blocos pequenos. Esse resultado ilustra de forma direta por que algoritmos sensíveis a I/O aleatório de pequeno tamanho de bloco apresentam degradação de desempenho desproporcional em relação a algoritmos com padrão de acesso sequencial, independentemente do nível de otimização aplicado à camada de memória.

É relevante observar que essa magnitude de diferença se torna ainda mais crítica em cenários de processamento paralelo: quando múltiplos núcleos do i5-13420H operam simultaneamente sobre um mesmo *dataset* – cenário comum em *workloads* de renderização ou compilação que utilizam os 12 *threads* disponíveis –, a demanda agregada de banda de memória aumenta proporcionalmente ao grau de paralelismo, mas a banda disponível do controlador de memória DDR5 permanece fixa. Isso significa

que, em *workloads* fortemente paralelos e com baixo reuso de cache por *thread*, o gargalo de desempenho pode migrar do processamento em si para a própria banda de memória compartilhada – um fenômeno conhecido na literatura como saturação de banda (*bandwidth saturation*), e que representa uma fronteira natural entre a investigação de hierarquia de memória conduzida neste estudo e a investigação de escalabilidade de paralelismo, não coberta neste trabalho mas discutida brevemente como direção futura na Seção 7.

5.4. Correlação com Especificações de Hardware

A Tabela 2 sintetiza a correlação entre as especificações de hardware identificadas e as observações experimentais obtidas.

Table 2. Correlação entre especificações de hardware e resultados experimentais

Especificação	Valor reportado	Observação experimental
Capacidade do L3	12 MiB	Queda de 17,7% no throughput ao exceder esse limiar
RAM	DDR5	Banda estabiliza em ≈ 8.900 MiB/s acima do L3
Prefetcher de hardware	Presente (Raptor Cove)	Razão B/A contida ($1,11\times$) frente ao esperado sem prefetcher
NVMe	SSD interno	Throughput sequencial $4,2\times$ menor que a RAM pós-L3

6. Discussão

Os resultados obtidos confirmam o modelo teórico da hierarquia de memória apresentado na Seção 3.2: a largura de banda efetiva é ditada pelo nível da hierarquia em que o *dataset* reside, e a transição observada é consistente com o comportamento previsto pela equação do AMAT quando a $Taxa_{miss}$ do L3 passa de aproximadamente zero para próxima de um. A proximidade entre o limiar observado (entre 16 e 128 MiB) e a capacidade nominal do L3 (12 MiB) não é coincidência, mas uma manifestação direta dessa capacidade – resultado que valida tanto o modelo teórico quanto a metodologia experimental empregada.

A razão B/A de $1,11\times$ no teste de localidade espacial é notavelmente baixa para um padrão de acesso que, segundo modelos simplificados de penalidade de *miss*, deveria gerar uma penalidade substancialmente maior em uma matriz column-major de 274,7 MB – um *dataset* mais de vinte vezes maior que o L3, portanto fora do alcance de qualquer nível de cache. A explicação mais plausível é que o *prefetcher* de hardware dos P-cores Raptor Cove é capaz de detectar *strides* regulares e antecipar os próximos acessos, transformando o que seria um *miss* em um acesso *prefetched* – efetivamente

ocultando parte da penalidade. Esse resultado tem implicação prática relevante: em arquiteturas com *prefetching* agressivo, a penalidade por acesso irregular é menor do que modelos didáticos simplificados sugerem, embora não seja eliminada por completo, como evidencia a diferença de 11% ainda mensurável entre os dois padrões de acesso.

A comparação com I/O de armazenamento reforça quantitativamente o motivo pelo qual a hierarquia de memória é estruturada em camadas: a diferença de quase duas ordens de grandeza entre a RAM e a leitura aleatória em NVMe demonstra que otimizações de localidade dentro da hierarquia de cache, embora relevantes (ganhos de até 17,7% neste estudo), operam em uma escala de problema distinta – e frequentemente menos crítica para o desempenho global de uma aplicação – do que evitar padrões de I/O aleatório de pequeno bloco em armazenamento persistente. Esse achado tem valor pedagógico: ilustra de forma concreta, com dados do mesmo sistema físico, a razão pela qual otimizações de cache e otimizações de I/O pertencem a ordens de grandeza distintas de impacto potencial sobre o desempenho de uma aplicação.

6.1. Heterogeneidade P+E e suas Implicações

Um aspecto da arquitetura híbrida que merece discussão à luz dos resultados obtidos é a assimetria de cache L2 entre P-cores e E-cores, identificada na Seção 3.2 e na Tabela 1: cada P-core possui 1.280 KB de L2 dedicado, enquanto os quatro E-cores compartilham um único bloco de 2.048 KB. Embora os experimentos de throughput conduzidos neste estudo não tenham distinguido explicitamente o tipo de núcleo em que o benchmark *mbw* foi executado – limitação reconhecida na Seção 6.2 –, essa assimetria sugere que o comportamento de cache observado pode variar dependendo de qual tipo de núcleo o escalonador do sistema operacional atribui à carga de trabalho. Em particular, é plausível que um *workload* executado preferencialmente em E-cores apresente uma transição de regime de cache em um ponto diferente do observado neste estudo, dado o L2 compartilhado entre quatro núcleos nesse conjunto. Essa hipótese não pôde ser testada diretamente com a metodologia empregada, mas representa uma direção natural para trabalhos futuros, discutida na Seção 7.

6.2. Ameaças à Validade

Seguindo a prática usual em estudos experimentais de sistemas, esta seção organiza as principais limitações do estudo em três categorias: validade interna, validade externa e validade de constructo.

Validade interna. A ausência de pontos de medição intermediários entre 16 MiB e 128 MiB impede a caracterização precisa da forma da curva de transição em torno do limiar de 12 MiB – não se sabe, por exemplo, se a transição é abrupta (um único patamar) ou gradual (uma rampa ao longo de alguns MiB adicionais). Adicionalmente, os testes de *mbw* não controlaram explicitamente a afinidade de núcleo (*CPU pinning*), o que significa que o escalonador do sistema operacional pode ter distribuído as execuções entre diferentes núcleos – potencialmente P-cores e E-cores em momentos distintos – introduzindo uma fonte de variância não quantificada nos resultados.

Validade externa. O ambiente de execução – WSL2 sobre Windows 11 – introduz uma camada de virtualização leve que pode adicionar *overhead* mensurável às operações de memória e de I/O, tornando os números absolutos reportados aqui conservadores em

relação a um ambiente Linux nativo (*bare-metal*). Os resultados quantitativos específicos (por exemplo, o valor exato de 8.900 MiB/s em regime de RAM) não devem, portanto, ser generalizados diretamente para outros ambientes de execução ou para outras unidades do mesmo modelo de processador sem nova validação; o padrão qualitativo identificado – a existência de um limiar de transição coincidente com a capacidade do L3 – é, no entanto, esperado ser reproduzível em qualquer ambiente com hierarquia de cache análoga.

Validade de construto. O uso do `mbw` como instrumento de medição de throughput captura especificamente o desempenho de operações de cópia de memória (`memcpy`), que não é necessariamente representativo de todos os padrões de acesso à memória encontrados em aplicações reais (por exemplo, acessos com razão leitura/escrita distinta, ou padrões com maior reuso de dados). Da mesma forma, o teste de localidade espacial implementado em Python puro introduz *overhead* de interpretação que pode mascarar parcialmente os efeitos de hierarquia de cache que estariam mais evidentes em código compilado (C ou C++), uma vez que o tempo de execução em Python é dominado, em parte, pelo custo do interpretador, e não exclusivamente pelo tempo de acesso à memória.

Por fim, a divergência inicial entre `lscpu/lsstopo` e o CPU-Z quanto à topologia de núcleos (Seção 4.1) ilustra um risco metodológico relevante para estudos similares: ferramentas de inspeção de hardware podem agregar ou simplificar a topologia real de formas que obscurecem a heterogeneidade de arquiteturas híbridas, exigindo validação cruzada entre múltiplas fontes antes de se reportar especificações de hardware como fato estabelecido.

6.3. Implicações Práticas para Desenvolvimento de Software

Os resultados obtidos permitem derivar algumas implicações práticas para o desenvolvimento de software que opera sobre hardware com características semelhantes às do processador estudado. Em primeiro lugar, a confirmação experimental de que a transição de regime de cache ocorre próxima à capacidade nominal do L3 reforça o valor prático de técnicas de *blocking* (ou *tiling*), amplamente empregadas em bibliotecas de álgebra linear de alto desempenho, como implementações de BLAS: ao particionar operações sobre grandes matrizes em blocos que cabem inteiramente no L3, é possível manter a $Taxa_{miss}$ próxima de zero por mais tempo durante a execução, evitando a penalidade de até 17,7% identificada neste estudo. Esse princípio é particularmente relevante para algoritmos de multiplicação de matrizes, processamento de imagens e outras operações com alto grau de reuso de dados.

Em segundo lugar, o resultado de que o *prefetcher* de hardware atenua substancialmente a penalidade de acessos em *stride* regular (razão B/A de apenas $1,11\times$) sugere que desenvolvedores podem, em certa medida, confiar no *prefetcher* para padrões de acesso previsíveis, mesmo quando não estritamente sequenciais – por exemplo, ao percorrer estruturas de dados com espaçamento fixo entre elementos relevantes. Esse comportamento não deve, entretanto, ser interpretado como uma justificativa para negligenciar a localidade espacial por completo: a diferença de 11% permanece mensurável, e tende a ser ampliada em cenários com maior pressão sobre o L3 – por exemplo, quando múltiplos *threads* concorrentes disputam o mesmo cache compartilhado, reduzindo a capacidade efetiva disponível para cada um e, com isso, a margem do *prefetcher* para operar antecipadamente.

Por fim, a disparidade de quase duas ordens de grandeza entre o throughput de memória e a leitura aleatória em armazenamento NVMe reforça que decisões de design relacionadas ao padrão de acesso a disco (por exemplo, em sistemas de banco de dados ou em motores de armazenamento de chave-valor) tendem a ter impacto sobre o desempenho global substancialmente maior do que otimizações finas de localidade de cache, devendo ser priorizadas quando ambas as oportunidades de otimização estiverem disponíveis simultaneamente em um mesmo sistema.

7. Conclusão

Este artigo caracterizou experimentalmente o efeito da hierarquia de cache no throughput de memória do processador híbrido Intel Core i5-13420H. Os experimentos confirmaram que a largura de banda efetiva é determinada pelo nível da hierarquia em que o *dataset* reside, com queda de 17,7% ao ultrapassar o limiar do L3 de 12 MiB – resultado que valida tanto o modelo teórico de AMAT apresentado na Seção 3.2 quanto as especificações reportadas pelo fabricante.

A análise de localidade espacial revelou que o *prefetcher* de hardware dos P-cores Raptor Cove atenua significativamente o impacto de acessos em *stride*, resultando em razão B/A de apenas $1,11\times$ – valor menor do que o esperado em arquiteturas menos agressivas em *prefetching*, e um achado que qualifica de forma importante o que modelos didáticos simplificados de penalidade de *miss* costumam prever para esse tipo de padrão de acesso. A comparação com I/O de armazenamento situou esses resultados em perspectiva mais ampla, evidenciando uma diferença de quase duas ordens de grandeza entre o throughput de memória e a leitura aleatória em NVMe – uma constatação com valor pedagógico direto sobre a importância relativa de diferentes camadas de otimização de desempenho.

Do ponto de vista metodológico, este trabalho também demonstrou a importância de validar especificações de hardware reportadas por diferentes ferramentas de inspeção (Seção 4.1) antes de assumi-las como corretas, e discutiu de forma explícita – algo pouco comum em estudos didáticos de arquitetura de computadores – as limitações impostas por um ambiente de execução virtualizado (WSL2) sobre a fidelidade de microbenchmarks de memória.

Como trabalhos futuros, destacam-se quatro direções principais. Primeiro, a repetição dos experimentos em ambiente Linux *bare-metal* permitiria quantificar precisamente o *overhead* introduzido pelo WSL2 sobre as medições de banda de memória e I/O, isolando esse fator de confusão identificado na Seção 6.2. Segundo, o mapeamento de pontos intermediários de tamanho de array (por exemplo, 8, 10, 12, 14, 16, 20, 32 e 64 MiB) permitiria caracterizar com maior resolução a forma exata da curva de transição no limiar do L3, determinando se essa transição é abrupta ou gradual. Terceiro, a extensão da metodologia para investigar explicitamente o comportamento de cache quando a execução é forçada, via afinidade de núcleo (*CPU pinning*), a P-cores ou a E-cores isoladamente permitiria testar diretamente a hipótese de assimetria discutida na Seção 6.1, quantificando se a transição de regime de cache ocorre em pontos diferentes dependendo do tipo de núcleo utilizado. Por fim, a reimplementação do teste de localidade espacial em uma linguagem compilada (C ou C++) eliminaria o *overhead* de interpretação do Python como fator de confusão, permitindo isolar com maior precisão o efeito puro de localidade

sobre o tempo de execução. Como quinta direção, mencionada na Seção 6, a investigação da saturação de banda de memória sob *workloads* altamente paralelos – explorando os 12 *threads* do i5-13420H simultaneamente sobre *datasets* que excedem a capacidade do L3 – permitiria caracterizar em que ponto a banda agregada de memória, e não a capacidade de processamento dos núcleos, passa a ser o fator limitante de desempenho.

References

- [1] Drepper, U. What every programmer should know about memory. *Red Hat Inc.*, 2007.
- [2] Hennessy, J. L. and Patterson, D. A. *Computer Architecture: A Quantitative Approach*. 6th edition. Morgan Kaufmann, Cambridge, 2019.
- [3] Molka, D., Hackenberg, D., Schone, R., and Muller, M. S. Memory performance and cache coherency effects on an Intel Nehalem multiprocessor system. In *Proceedings of the 18th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, IEEE, 2009.